Webster 2026

Team: SANJI SIMP SQUAD

Team ID

404

Team Info

Member 1: Shraddha Sharma (20243266, CSE, Shradify_)

Member 2: Rudransh Pratap Singh (20243243, CSE, notr8dra)

Member 3: Stuti Agarwal (20243280, CSE, stutiagarwal_35955)

Description of the problem

High-demand digital assets and limited-edition inventory sell out in seconds, and the secondary market that follows is dominated by predatory scalpers and automated bot swarms. Today's resale channels fail in four core ways:

1. **Uncapped prices:** a face-value pass is relisted at several times its price.

2. **Unfair speed advantage:** bots with faster connections win every contested purchase.

3. **Unsafe settlement:** read-modify-write balance updates under concurrency can double-spend, oversell, or leave money in limbo.

4. **Opaque custody:** users must simply trust that the platform holds their money and tickets.

Darkhold Exchange is a secondary pass and relic marketplace built like an exchange. It combines an in-memory FIFO limit order book, price collars tied to face value, an atomic double-entry ledger, a Dutch-auction liquidation engine, and escrow with a cryptographic proof of solvency.

Fairness and safety are enforced by the architecture instead of by policy.

Website name: Darkhold Exchange

Process flow of the website

1. Onboarding. The user signs up and the account is bound to a verified identity. Per-identity holding limits are attached, and the wallet is funded (simulated top-up).

2. Primary drop (optional). For a hot release, the platform runs a frequent batch auction. Orders are collected during a time window, optionally as commit-reveal hashes, and cleared at one uniform price so speed confers no advantage.

3. Browse markets. Each event tier (e.g. Day-1 General) is a market. The user sees a live Level-2 ladder, depth chart, trade tape, face value, and the current price collar.

4. Place an order. The client validates the order against the collar. The gateway authenticates the user, applies rate limits and proof-of-work, and pre-checks the collar. The command then enters a lock-free ring and receives a global sequence number.

5. Reserve and match. The core reserves cash (buy) or the pass (sell) so settlement cannot fail mid-match, then matches by price-time priority. Limit orders may rest in the book, and market orders sweep within a protected price range.

6. Settle. Every fill posts a balanced double-entry journal entry atomically, moving cash and pass together, with fees to a platform account. Optional escrow mode locks funds until a signed delivery receipt is verified, and refunds on timeout.

7. Live updates. Book changes, fills, and balance updates stream over WebSockets as sequenced diffs. The client detects gaps and resyncs from a snapshot.

8. Distressed inventory. Unsold or low-demand inventory is liquidated through a Dutch auction whose price decays on a published schedule. The first valid acceptance wins, and the auction clears when demand meets supply.

9. Verify. Users open the proof page to check their balance's inclusion in the published Merkle-sum solvency root, and to verify the hash-chained audit log, all computed in the browser.

FEATURES ANNOUNCEMENT

• **In-memory matching engine:** FIFO price-time priority, limit and market orders, cancel, self-trade prevention.
• **Real-time market depth:** live L2 ladder and Canvas depth chart over WebSockets, with no page reload.
• **Anti-scalping price collars**: maximum and minimum prices derived from the original face value, enforced server-side, plus per-account holding limits.
• **Atomic trade settlement:** double-entry ledger with integer minor units, idempotency keys, and continuously checked conservation invariants.
• **Dutch auction engine:** price-decay schedule, real-time rounds, and an atomic first-accept (compare-and-set) rule.
• **High-concurrency resilience:** target of 1,000+ orders/s using lock-free rings in front of a single-writer matching core, with k6 and Locust benchmarks.
• **Escrow and proof of solvency:** escrow state machine with signed delivery receipts, and a Merkle-sum tree of balances verifiable in the browser.
• **Frequent batch auctions** with commit-reveal for hot drops, and a hash-chained public audit log.
• **Scalper-bot simulator** that demonstrates attack, defence, and measured outcome.
• **Live metrics and latency panel** (sent → acknowledged → visible).
• **Order book replay**: scrub backwards through time to see how the book and price evolved (the ordered order log makes this a simple replay).
• **Bots vs humans simulation**: a switch that releases bot traffic with and without the collar and rate limits, showing what the protections prevent.

External libraries/APIs

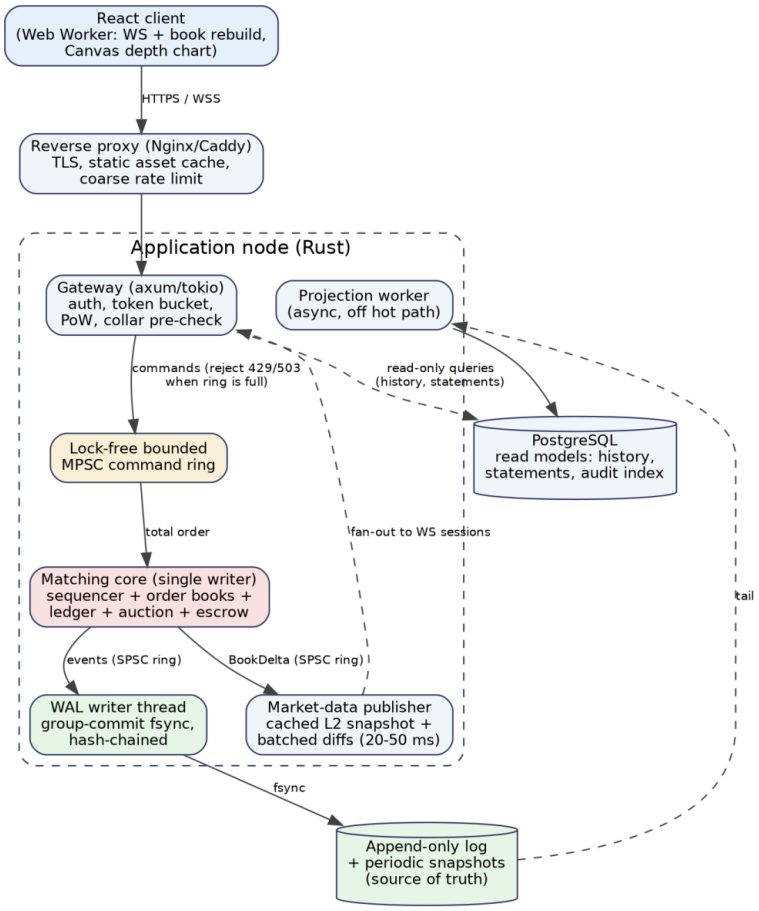
Library	Role	Why this over alternatives	Backing / licence
Rust (stable)	Engine, ledger, gateway	No garbage-collector pauses, so tail latency is predictable; ownership rules eliminate data races at compile time, which directly supports the zero-race-condition goal. Go is simpler but its GC adds latency jitter, and Node.js is single-threaded for CPU work.	Rust Foundation / community; MIT + Apache-2.0
Tokio + axum	Async runtime, HTTP and WebSocket gateway	axum is built by the Tokio team on tower and hyper, with first-class WebSocket and middleware support, so rate limiting and auth are composable. Preferred over actix-web (separate actor-style abstractions) and warp (filter-style API that is harder to maintain).	tokio-rs organization, widely used in production; MIT
crossbeam-queue	Lock-free bounded queues (ring)	Mature, audited lock-free bounded MPMC/MPSC queue. Preferred over hand-rolled atomics (high bug risk) and Redis Streams (adds a network hop and an extra process on the hot path).	Rust community, long-lived; MIT / Apache-2.0
sha2 (RustCrypto), ed25519-dalek	Hash chain, Merkle tree, signed receipts	Pure-Rust, widely reviewed implementations. Preferred over OpenSSL bindings (heavier build and linking) and any custom cryptography.	RustCrypto project; MIT / Apache-2.0, BSD-3
rmp-serde / @msgpack/msgpack	Binary wire format	Compact, fast to parse in the browser, and needs no code-generation step. Preferred over JSON (larger, slower to parse at tick rates) and Protobuf (extra tooling, little benefit at our message shapes).	Open spec with broad implementations; MIT / Apache-2.0
ts-rs (or specta)	Generate TypeScript types from Rust structs	Keeps the client and server protocol from drifting apart, instead of maintaining two hand-written definitions.	Community; MIT
PostgreSQL + sqlx	Read models: history, statements	ACID, mature, relational queries for statements and audit lookups. Preferred over MongoDB (weaker multi-row consistency for financial history) and SQLite (single-writer limits under concurrent reads). Used off the hot path only.	PostgreSQL Global Development Group; PostgreSQL licence (permissive)

React + TypeScript + Vite	Frontend	Largest ecosystem and hiring-market recognition. Vite gives fast builds and HMR, and replaces the deprecated Create React App. The hot path stays outside React rendering.	Meta + community; MIT. Vite: community and VoidZero; MIT
Zustand	Client state store	Store lives outside the React tree, with selector subscriptions that avoid re-render storms. Preferred over Redux Toolkit (more boilerplate) and Context (re-renders every consumer).	Poimandres collective; MIT
Comlink	Web Worker RPC	Moves WebSocket parsing and book reconstruction off the UI thread with a small proxy API, instead of hand-written postMessage plumbing.	Google Chrome Labs; Apache-2.0
uPlot / lightweigh t-charts	Streaming price charts	Both are far faster at streaming updates than general charting libraries such as Chart.js or Recharts. The depth chart itself is custom Canvas.	Independent maintainer (uPlot, MIT); TradingView (lightweight-charts, Apache-2.0)
proptest, Criterion, hdrhistogra m	Property tests, micro-benchm arks, latency percentiles	Property testing finds edge cases that example tests miss. HdrHistogram reports tail percentiles correctly, where simple averages hide them.	Rust community; MIT / Apache-2.0
Vitest, fast-check, Playwright	Frontend unit, property, and end-to-end tests	Vitest integrates directly with Vite. fast-check verifies that the book-sync logic converges under arbitrary drops and reorderings. Playwright drives real browsers.	Community; MIT. Playwright: Microsoft; Apache-2.0
k6 + Locust	Load generation	k6 supports WebSocket and arrival-rate (open-model) executors that avoid coordinated omission. Locust adds Python-scripted HTTP user flows. Preferred over JMeter (heavier and GUI-centric). k6 is used as an external tool, not linked into our code.	k6: Grafana Labs; AGPL-3.0. Locust: community; MIT
Nginx / Caddy	Reverse proxy, TLS, static asset caching	Offloads TLS and static files from the Rust node and applies a coarse first-line rate limit.	Nginx: F5 / community, BSD-2; Caddy: community, Apache-2.0

Core Engineering Architecture

1. Infrastructure layout and data flow

The system is split so that the one part that needs strict ordering (matching and settlement) is a single, deterministic, non-blocking thread, while everything slow or parallel (network, disk, database, fan-out) sits behind queues around it.



Write path: what happens when a user places an order

1. Edge: the proxy terminates TLS, enforces request-size limits, and applies a coarse rate limit. Static assets are served from cache and never reach the application node.

2. Gateway: authenticates the session (JWT), applies per-identity and per-IP token buckets and a proof-of-work challenge, validates the schema, and runs a cheap collar pre-check. Obviously invalid or abusive requests are dropped here, before they cost the core anything.

3. Ring: the command, carrying its idempotency key, is pushed into a bounded lock-free MPSC ring. If the ring is full, the gateway returns 429/503 with Retry-After immediately, instead of queueing without limit.

4. Core (single writer): pops commands in one total order, stamps a sequence number and timestamp, checks ownership or funds, reserves them, matches, and posts balanced journal entries. It performs no network or disk I/O and no heap allocation on the hot path.

5. Log: the core hands events to a WAL thread through an SPSC ring. The WAL batches records (group commit), fsyncs, and links each record to the previous one with a hash. Because the core is deterministic, replaying the logged commands reproduces the exact state, which also powers the order-book replay feature.

6. Fan-out: book deltas go to the market-data publisher, which updates a cached L2 snapshot and pushes batched diffs to WebSocket sessions. A projection worker tails the log and asynchronously updates PostgreSQL read models.

Read path, caching, and storage

Data	Where it lives	Why
Live order books, balances, auctions	In-memory structures owned by the core thread	No locks and no database round-trip on the hot path; O(1) operations
Source of truth	Append-only hash-chained log + periodic snapshots	Durable, auditable, and replayable; recovery = latest snapshot + log tail
L2 snapshot for new/reconnecting clients	Cached copy inside the publisher	A reconnect storm never touches the engine
History, statements, audit index	PostgreSQL read models (async projection)	Heavy reads are isolated from trading; a database outage does not stop trading
Static frontend	Proxy cache / CDN	Removes asset traffic from the application node

Algorithmic complexity

Operation	Our design	Naive baseline (sorted list / tree)
Add limit order	O(1): index into a flat price-level array (the collar bounds the price range)	O(log P) tree insert, or O(N) list insert
Cancel order	O(1) via order-id to slab handle and intrusive doubly linked list	O(N) scan, or O(log P) plus search
Best bid / ask	O(1) pointer, with a bitset scan when a level empties	O(log P) or O(1) with extra upkeep
Match	O(k) for k fills, plus O(levels exhausted)	O(k log P)

2. Surviving a rush: hundreds of users, one deadline

• **Typical failure moment**: a drop opens at a fixed time and hundreds of users, plus bots, submit heavy actions at once. The plan, by layer:

• **Admission control before the engine**: per-identity and per-IP token buckets, a global limit, and adaptive proof-of-work that makes mass automated submission expensive for bots but cheap for a human.

• **Bounded queues and back-pressure**: the command ring has a fixed size. Overload becomes a fast, explicit rejection with Retry-After rather than rising latency, memory growth, and a crash.

• **Flatten the herd**: for hot drops, a frequent batch auction (with commit-reveal) turns millions of racing requests into one clearing computation per window, and removes the incentive to be first.

• **Isolation of slow work**: disk (WAL thread), database (projection worker), and WebSocket fan-out each run on separate threads behind rings, so a slow disk, slow database, or slow client cannot stall matching. Database writes are batched off the hot path with pooled connections (sqlx pool), so thousands of simultaneous requests cannot overwhelm PostgreSQL.

• **Slow-consumer protection**: diffs are batched every 20–50 ms and each client has a bounded send buffer. A client that falls behind is resynced from the cached snapshot or disconnected, not buffered indefinitely.

• **Recovery**: after a crash, the node loads the latest snapshot and replays the log tail to a bit-identical state. Docker restart policies and health checks bring the node back automatically, and reads are served from the cached snapshot while the engine recovers. Warm standby (a second node tailing the log) is a planned stretch goal and is described as such, not as a delivered feature.

• **Targets**: 1,000+ orders sustained, engine p99 in the microsecond range, and end-to-end p99 under 10 ms at 1,000 orders.

3. Why this is production-grade, not a classroom assignment

Each block compares the naive approach, our approach, and the justification.

A. Trade settlement and balances

• **Naive**: Per request: SELECT balance, check in application code, then separate UPDATEs for buyer, seller, and ticket. Two concurrent buys both read a balance of Rs. 5,000, both pass the check and both commit, so the balance goes negative (lost update, double-spend). Two sells match the same bid and one ticket is sold twice. A crash between debiting the buyer and crediting the seller makes money vanish. On a flaky network the client retries and creates a duplicate order.

• **Ours**: Single-writer core. Funds or passes are reserved before the order reaches the book. Each fill posts one balanced journal entry (cash and pass legs, per-asset sum = 0), guarded by an idempotency key, applied atomically in one step. The ledger is append-only, and the database layer adds CHECK constraints (balance >= 0), unique idempotency keys, and no UPDATE or DELETE on ledger rows as a second line of defence.

• **Why it wins**: A concurrent read-modify-write cannot happen, since there is only one writer. Money cannot be created or lost, which is machine-checked by invariants (global conservation, no negative balances, held = open reservations) and by the continuous reconciliation job. Retries are safe.

B. Concurrency control

• **Naive**: Global mutex or database row locks around the order book, or a shared array edited by many handlers, with every order matched through database queries. Contended locks serialize threads, add context-switch latency, and risk deadlock; matching through the database adds tens of milliseconds per order and collapses at about 1,000 orders per second.

• **Ours**: Many gateway threads feed a lock-free bounded MPSC ring. One core thread owns the books (Disruptor / single-writer pattern), with separate SPSC rings out to the log and publisher.

• **Why it wins**: Zero data races by construction, one deterministic total order of events, and microsecond-scale matching. Lock-free structures are used where contention exists (the ring) and avoided where a single owner is simpler and faster.

C. Live market data

• **Naive**: Clients poll an HTTP endpoint, or the server re-sends the whole book on every change to every client. Bandwidth grows with clients × book size, updates can arrive out of order, and a missed message silently corrupts the displayed book.

• **Ours**: Snapshot with sequence number, then sequenced incremental diffs batched every 20–50 ms in a binary format. Gap detection triggers resync from the cached snapshot. The client rebuilds the book in a Web Worker.

• **Why it wins**: Bandwidth proportional to changes only, and a consistent book guaranteed by sequencing. Reconnect storms hit a cache, not the engine, and the UI thread stays smooth.

D. Dutch auction acceptance

• **Naive**: Check-then-update: read remaining quantity, then decrement. Two buyers racing for the last unit both see it available, so the item is oversold. Price streamed tick by tick creates load and clock drift.

• **Ours**: Acceptance is a command processed by the core: a compare-and-set on auction state and remaining quantity in the total order. The price is a published function of time, computed locally on the client from a measured server-clock offset.

• **Why it wins**: Overselling is impossible, ties are broken deterministically by sequence number, and the price stream costs no bandwidth and can be independently verified against the formula.

E. Anti-scalping and anti-bot

• **Naive**: Price and bot checks only in the UI, or a CAPTCHA on login. A script calls the API directly with any price, and many accounts bypass any per-session limit.

• **Ours**: Server-enforced collars tied to face value, identity-bound holding limits, per-user and per-IP rate limits and proof-of-work, frequent batch auctions with commit-reveal, and a public hash-chained audit log.

• **Why it wins**: Gouging is structurally impossible, not just discouraged. Speed advantage is removed within a batch window. Fairness is checkable by anyone. The bots vs humans simulator reports what each defense blocks, with measured numbers.

F. Trust in custody

• **Naive**: Show a balance from a database table and ask users to trust it.

• **Ours**: Merkle-sum tree over salted user balances with a published root and total, verified in the user's browser, plus a continuous ledger reconciliation job. Escrow releases funds only against a signed delivery receipt and refunds on timeout.

• **Why it wins**: Users can verify their own balance is included in the liabilities total, and funds cannot be released without delivery evidence. (Reserves are simulated, and this is stated openly.)

Verification plan (how we prove it)

• **Property-based tests:** the book never crosses, price-time priority holds, quantity is conserved, no balance goes negative, and the ledger sums to zero for any random (including concurrent) order stream.

• **Differential testing:** the same streams run through a deliberately simple reference matcher and our engine, and outputs must match exactly.

• **Deterministic replay**: replaying the log must reproduce an identical state hash.

• **Load tests**: k6-based HTTP load tests with configurable arrival rates, measuring throughput, p50/p99 latency, and rejection rates. Tests target 1,000 order submissions/s where hardware permits, with pass/fail thresholds in CI and benchmark scripts/results in the repository. WebSocket load testing is an optional extension.

Class diagram

1. (Initial)(Open to changes): class.png